

Angular Avançado · Módulo 1

Sessão 12

Arquitetura: domains, layers e o Sheriff

João Gonçalves · joaogoncalves.net · Making the complicated simple

Onde ficámos na Sessão 11

O FlightEdit completo — e onde ficou cada peça

```
domains/
├── ticketing/
│   ├── feature-booking/flight-edit/   ← o form, o submit
│   └── data/
│       ├── flight-schema.ts           ← validators + schema logic
│       └── flight-validators.ts       ← From ≠ To
└── shared/
    └── ui-forms/validation-errors/
        └── validation-pane.ts         ← mostra erros
```

Pergunta

- Porquê **aqui**?
- E se o pane estivesse em `ticketing/ui`?
- E o schema dentro da feature?

Hoje: dar nome às **regras** que já estávamos a seguir — e pôr uma ferramenta a verificá-las.

Agenda de hoje

01	**Porquê arquitetura?**	O problema · vertical slicing · fronteiras	≈ 30 min
02	**Layers e dependency rules**	A architecture matrix · o Skyboard na matrix	≈ 25 min
03	**Sheriff**	As dependency rules verificadas no Skyboard	≈ 40 min
04	**Na vida real**	Feature slicing · stores · modulith · resumo	≈ 20 min



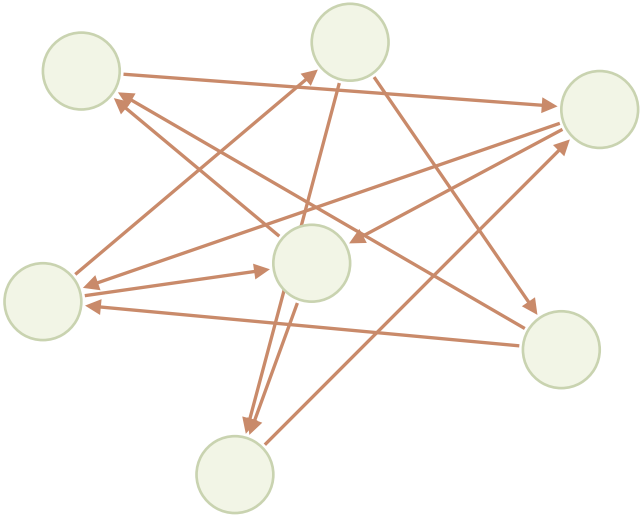
Bloco A

Uma app que cresce

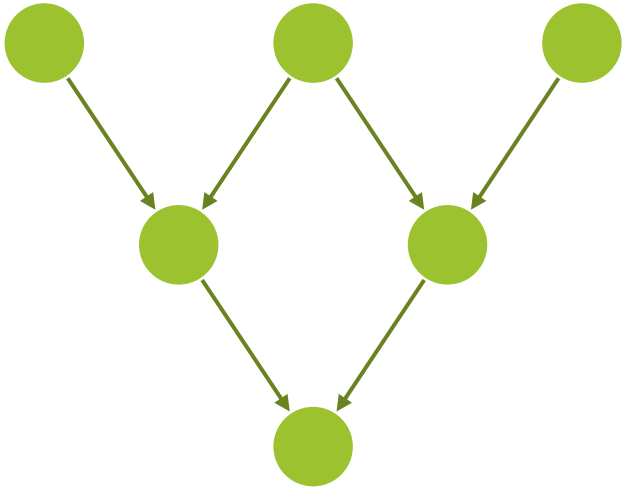
O problema, vertical slicing e onde cortar

O problema: uma app que cresce

Sem regras



Com regras: só num sentido



O custo de uma app não está em **escrevê-la** — está em **mudá-la** durante anos.

Quatro sintomas

Ciclos

Tudo depende de tudo: imports em todas as direções.

Mudar A parte B

Uma alteração na reserva rebenta o check-in.

Onde ponho isto?

Cada um decide à sua maneira; a estrutura degrada-se.

Equipas a colidir

Os mesmos ficheiros, conflitos, coordenação.

Qual destes vos custa **mais** no dia a dia?

Cortar pelo negócio: vertical slicing

Horizontal (por tipo técnico)

`components/` booking · checkin · boarding misturados

`services/` booking · checkin · boarding misturados

`models/` booking · checkin · boarding misturados

Vertical (por assunto do negócio)

booking

components
services
models

checkin

components
services
models

boarding

components
services
models

Para mudar **um** caso de uso, mexe-se numa **fatia** — não em todas as pastas.

Porquê: cinco objetivos

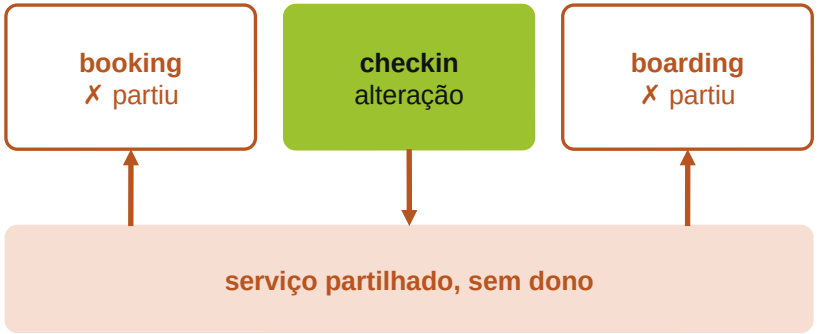
Low coupling	Mudar uma vertical não parte as outras
High cohesion	O que muda junto, fica junto
Conway's Law	O sistema copia a organização → desenhar as equipas a pensar na arquitetura (Inverse Conway Maneuver)
Autonomous teams	Uma equipa, uma fatia: entregas independentes
Reduced cognitive load	Só é preciso ter na cabeça a sua parte

Um a um nos próximos cinco slides.

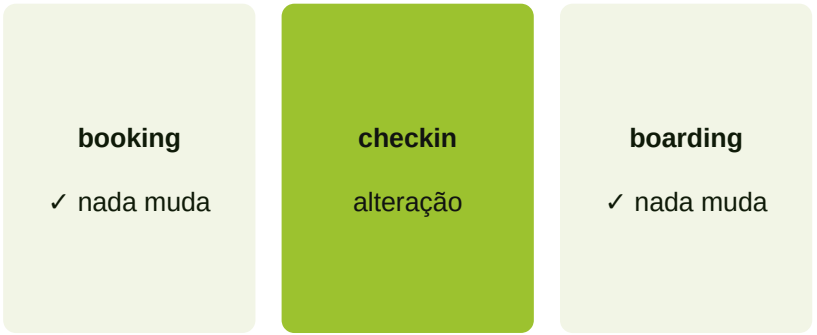
1 · Low coupling

Mudar uma vertical não parte as outras

Tudo ligado



Verticals



Independentes

Cada vertical desenvolve-se, testa-se e (idealmente) faz deploy **à parte**.

Dois lados da mesma moeda

Coupling olha para **fora** da fatia; cohesion olha para **dentro** (a seguir).

2 · High cohesion

O que muda junto, fica junto

Pedido: "no **FlightEdit**, a data de regresso tem de ser **depois** da ida"

Por tipo técnico → 4 pastas

```
models/        flight.ts ✎   passenger.ts
validators/    flight-validators.ts ✎   …
components/    flight-edit/ ✎   flight-card/   …
services/      flight-client.ts ✎   auth-store.ts

quem mais usa o flight-validators.ts?
```

Por negócio → 1 fatia

```
ticketing/
├── feature-booking/flight-edit/   ✎
└── data/
    ├── flight.ts
    ├── flight-schema.ts           ✎
    └── flight-validators.ts       ✎
```

Perceber

Lê-se uma fatia, não a app.

Menos erros

Vê-se tudo o que a alteração toca.

Mudar

Alteração contida, PR pequeno.

3 · Conway's Law

O sistema copia a organização que o faz

Equipas por tecnologia

Frontend

Backend

Base de dados

cancelar reserva: 3 equipas, 3 backlogs

Inverse Conway Maneuver

equipa booking

frontend
backend · testes

equipa checkin

frontend
backend · testes

equipa boarding

frontend
backend · testes

cancelar reserva: 1 equipa

Duas regras

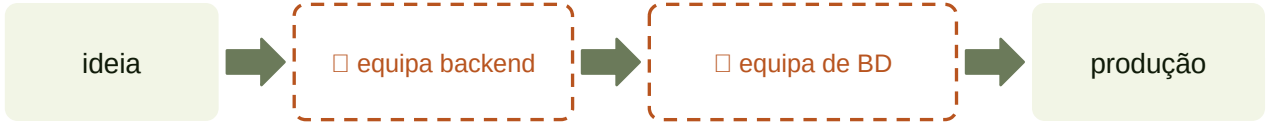
- Uma equipa → uma ou **várias** verticals
- Uma vertical → **uma só** equipa (ownership)

Querem saber como vai ser a arquitetura daqui a três anos? **Olhem para o organigrama.**

4 · Autonomous teams

Uma equipa, uma fatia — do princípio ao fim

Com hand-offs



Equipa autónoma



Funcional e técnica

Dona do **quê** (regras) e do **como** (código, testes, deploy).

Entregas independentes

Modulith: decide e desenvolve à parte.
Micro frontends: também o deploy.

Só com boas fronteiras

Verticals mal cortadas → as equipas voltam a depender umas das outras.

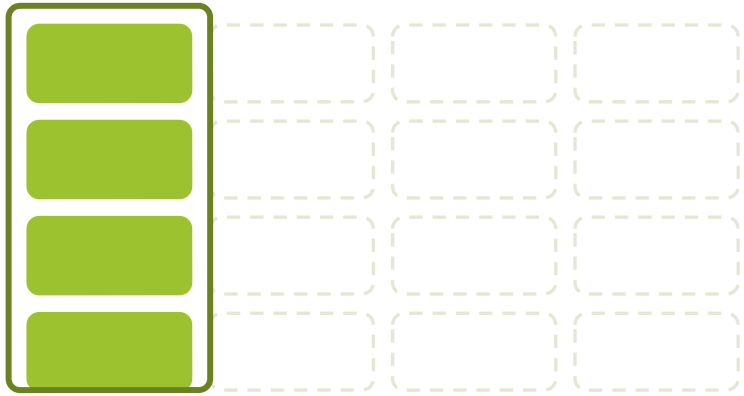
5 · Reduced cognitive load

O que tens de ter na cabeça para mudar sem partir

Sem fronteiras: a app toda na cabeça

"se mexer aqui, quem mais usa isto?"

Com fronteiras: só a tua fatia



fronteira garantida: dependency rules + Sheriff

Uma boa arquitetura não é a que tu percebes toda. É a que te deixa **trabalhar sem precisares de a perceber toda**.

Foco · onboarding mais rápido · menos bugs — pesa sobretudo em sistemas **complexos**.

De onde vêm as heurísticas: DDD

Domain-Driven Design · Eric Evans, 2003 · desenhar a partir do negócio

Strategic design
o mapa grande: dividir e relacionar as partes

← **hoje**

Tactical design
entities · aggregates · repositories… (sobretudo backend)

`Domain: transporte aéreo`

booking	**checkin**	**boarding**
flight = oferta	flight = lista de passageiros	flight = porta, hora

Ubiquitous language

A linguagem do negócio — também no código.

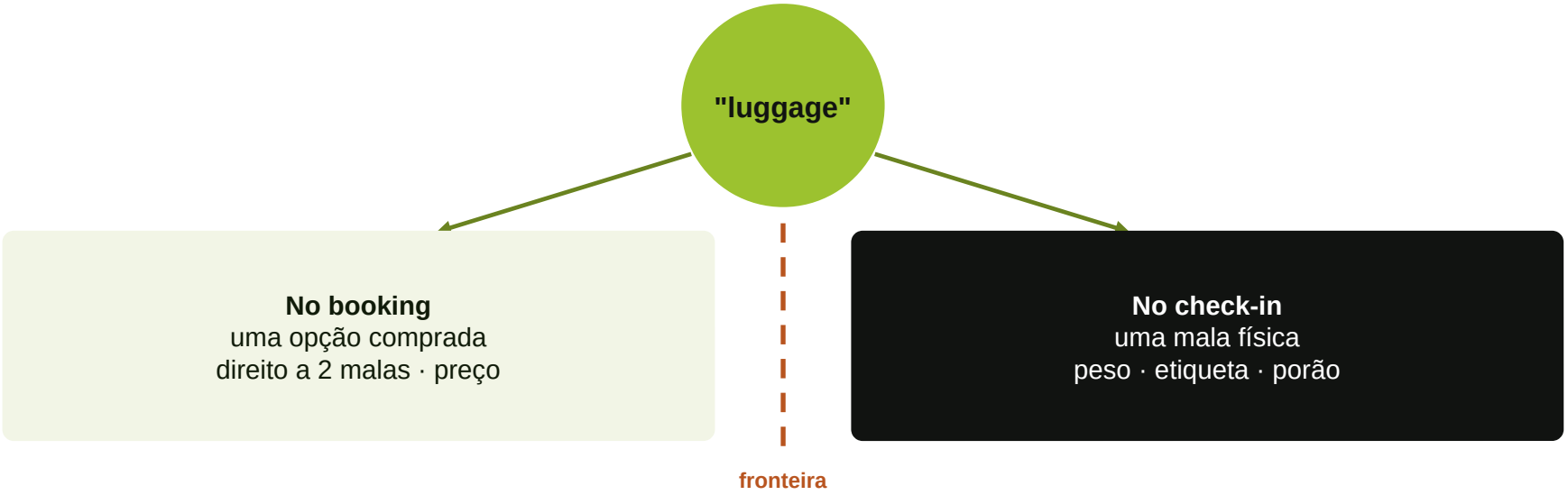
Bounded context

Onde um modelo e uma linguagem são válidos.

No Skyboard: cada pasta em `domains/` ≈ um bounded context. A pergunta: **onde** traçar as fronteiras?

Heurística 1 · Language

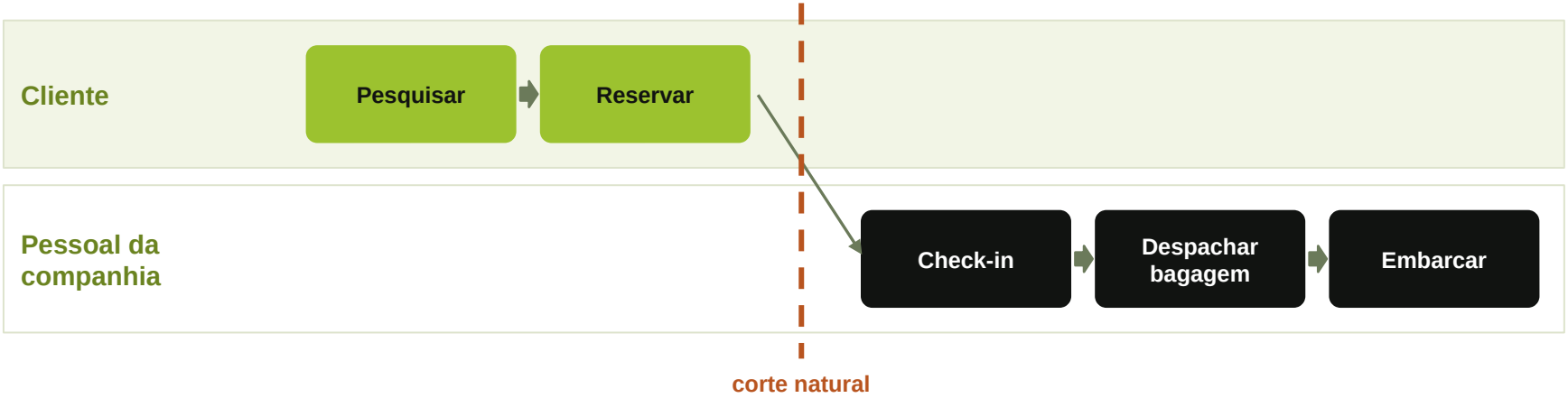
Mesmo termo, significado diferente → fronteira



Mesmo termo, **mesmo** significado → mesmo contexto. Significado **diferente** → contexto diferente.

Heurística 2 · Responsibilities

Muda quem é responsável → muda o modelo



Porquê?

Responsáveis diferentes → termos, regras e modelo diferentes.

Liga ao Conway

Responsáveis diferentes são, muitas vezes, **equipas** diferentes.

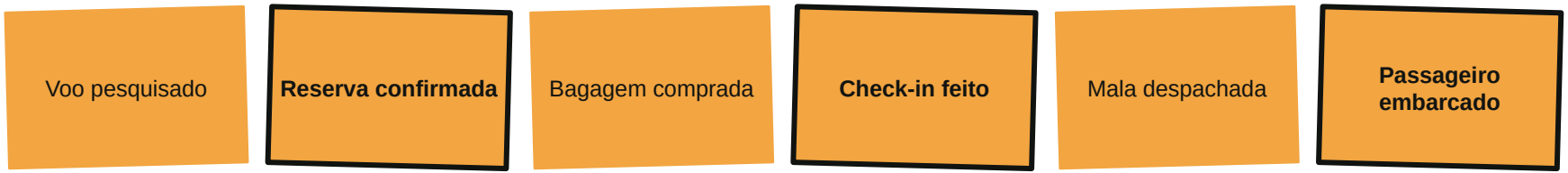
Heurística 3 · Pivotal events

Pontos de viragem difíceis de desfazer



Depois de cada um, as regras mudam — como uma mudança de cena num filme.

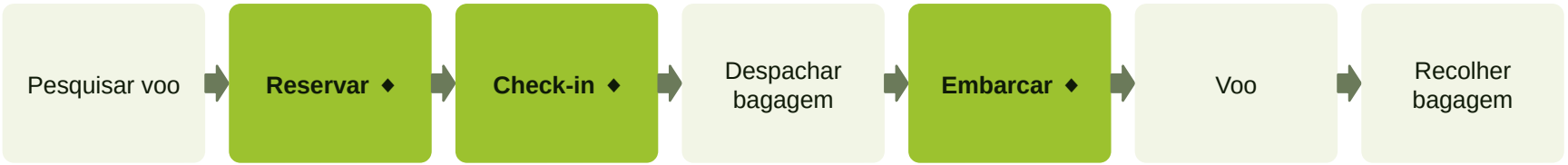
Event Storming: negócio + técnicos, post-its numa parede



As três heurísticas podem **contradizer-se**: decide-se — e assume-se a decisão.

Exercício: onde cortariam?

O processo de uma viagem



◆ pivotal event

Perguntas

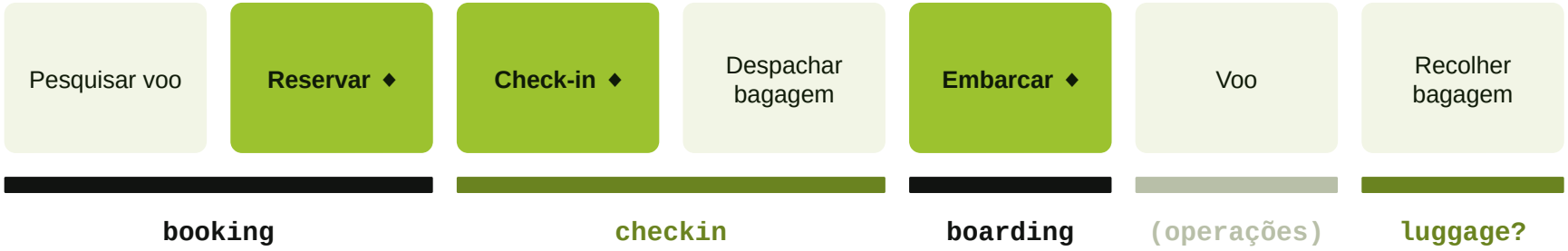
- Quantos domains? Onde ficam as fronteiras?
- Que heurística usaram para cada corte?

E no Skyboard?

- Hoje: **um** domain (`ticketing`) com `feature-booking` e `feature-boarding`
- O boarding devia ser um domain próprio?

Uma resposta possível

Não a única — o que importa é decidir conscientemente



Que heurística?

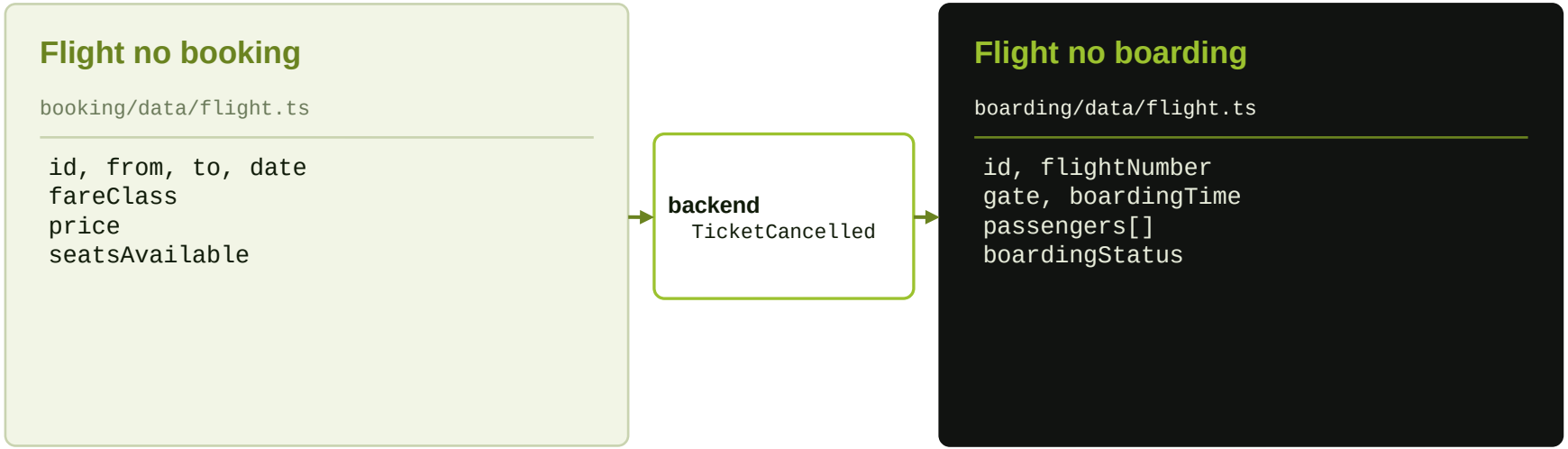
- **Responsibilities**: o cliente reserva · agentes do check-in ao embarque
- **Language**: "luggage" muda de significado
- **Pivotal events** fecham cada parte

E no Skyboard?

- Um domain, `ticketing`, com duas features
- Se o boarding crescer → domain próprio

Um "Flight" não é um "Flight"

Cada contexto tem o seu modelo



Um só modelo para tudo → campos irrelevantes, validações frágeis, mudanças que partem o outro lado.

No Skyboard: um `Flight` para booking e boarding — chega, porque a app é pequena.

Frontend e backend: o mesmo corte?

Normalmente, sim

- As fatias do frontend seguem as do backend
- Equipas que entregam valor de ponta a ponta

Às vezes, não → BFF

- **Backend for Frontend**: traduz a linguagem do backend
- Está no backend, pertence ao frontend
- Também: cache, segurança, monitorização



Bloco B

Domains × layers

A architecture matrix — e quem pode usar quem

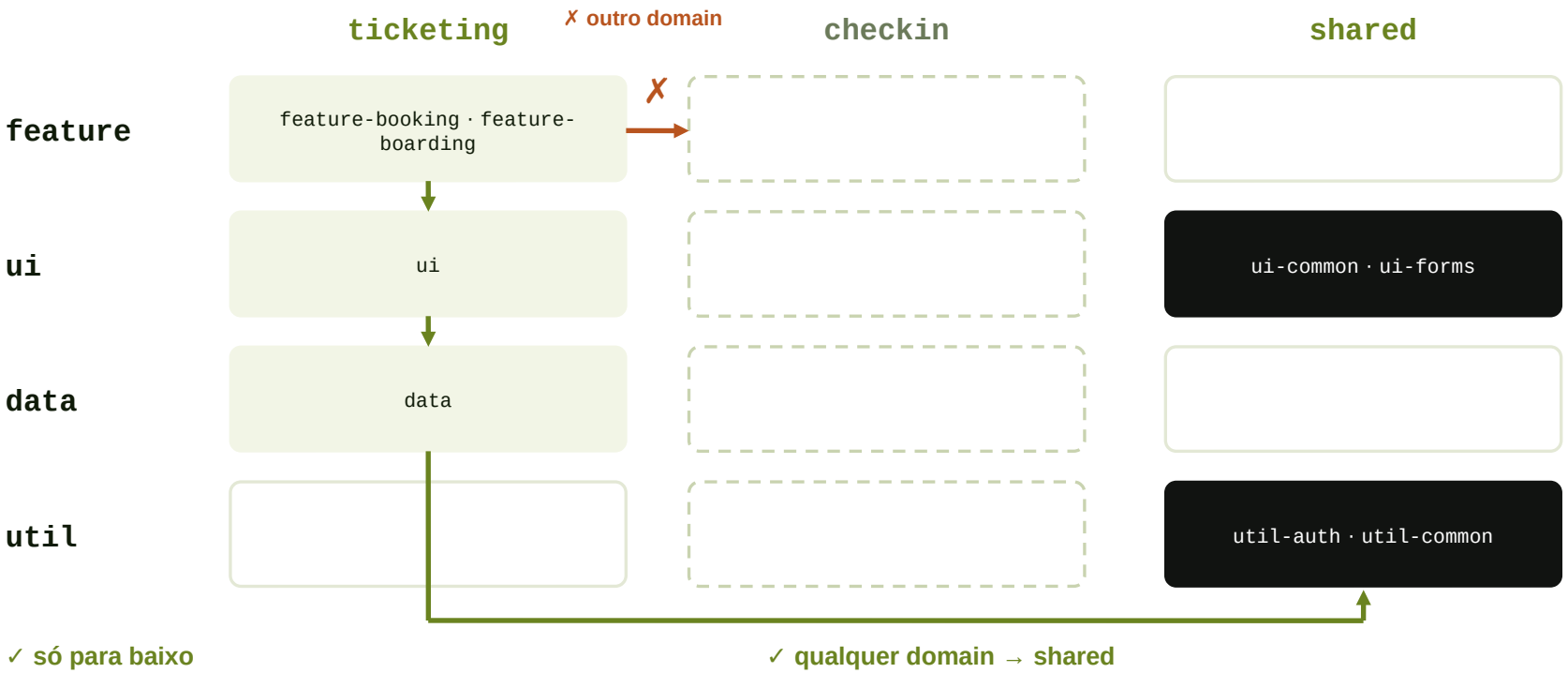
As layers: uma pilha



feature	Casos de uso · smart components	`FlightSearch` · `FlightEdit` · `BoardingList`
ui	Dumb components · só `input/output`	`FlightCard` · `PassengerRow` · `ValidationPane`
data	Modelo · backend · validators · state partilhado	`Flight` · `FlightClient` · `flightSchema`
util	Genérico, sem negócio	`AuthStore` · `ConfigService`

`shared`: o domain que todos podem usar — sobretudo **ui** e **util** (código técnico).

A architecture matrix: cada célula é uma pasta



Três dependency rules

importa →	`feature`	`ui`	`data`	`util`
`feature`	—	✓	✓	✓
`ui`	✗	—	✓	✓
`data`	✗	✗	—	✓
`util`	✗	✗	✗	✗ ?

Domains

O próprio + `shared`. Nunca outro domain diretamente.

API pública

Sem `index.ts`: tudo é público **exceto** a subpasta `internal/`.

A linha importa a coluna. `ui → data` ✓ (o `FlightCard` recebe um `Flight`) · `ui → feature` ✗.

O Skyboard na architecture matrix

Organizado assim desde a Sessão 1

	ticketing	shared
feature	`FlightSearch` (+ `FlightStore`) · `FlightEdit` · `BoardingList` · `BoardingLocked`	
ui	`FlightCard` · `PassengerRow`	`SimpleDelayStepper` · `ValidationPane` `loggingInterceptor ?`
data	`Flight` · `FlightClient` · `flightSchema` · `validateRoundTrip`	
util		`AuthStore` · `authGuard` · `authInterceptor` · `ConfigService` · `exitGuard`

A `shell` (navbar, home, about) fica fora: é a raiz e pode usar tudo.

Exercício: onde ponho isto?

Domain + layer

1. Um componente de calendário genérico
2. A `FlightStore`, se o boarding também precisar dela
3. Um pipe que formata durações ("2h 15m")
4. Um interceptor que trata 401/403
5. A página "As minhas reservas"
6. O tipo `Passenger`
7. Um `PassengerCard` usado só no boarding
8. Um serviço de traduções (i18n)

Respostas possíveis

1	Calendário genérico	`shared · ui`
2	`FlightStore` partilhada	`ticketing · data`
3	Pipe de durações	`shared · util (ou ui)`
4	Interceptor 401/403	`shared · util-auth`
5	"As minhas reservas"	`ticketing · feature`
6	O tipo `Passenger`	`ticketing · data`
7	`PassengerCard` só no boarding	`na feature-boarding`
8	Traduções (i18n)	`shared · util`

A **layer** quase nunca é a dúvida — a dúvida é o **domain** e **local vs partilhado**.



Bloco C

As regras, verificadas

Dependency rules no papel esquecem-se — uma ferramenta lê os imports

O que já é verificado — e o que não é

Ferramentas que verificam regras por nós

TypeScript ✓

verifica os tipos

string ≠ number

ESLint ✓

verifica o estilo

variável não usada

??? ✗

quem pode importar quem?

ninguém verifica

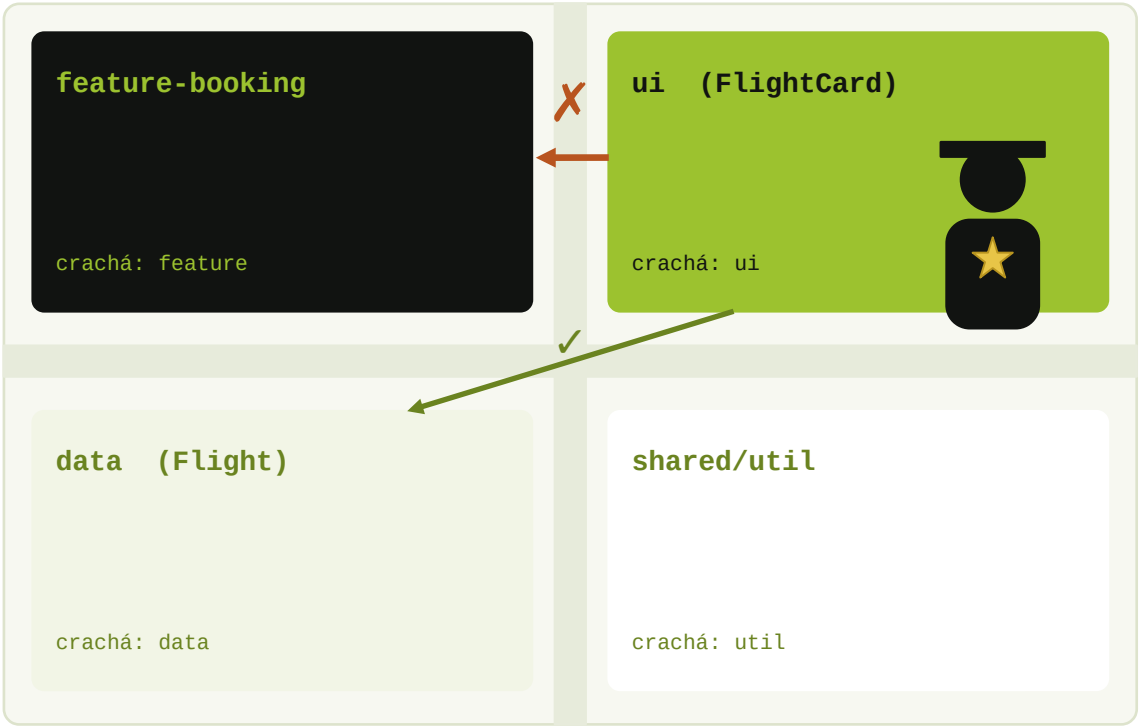
```
// ticketing/ui/flight-card/flight-card.ts — um componente de apresentação
import { FlightStore } from '../../feature-booking/flight-search/flight-store';
```

Compila ✓ Funciona ✓ Code review… ninguém deu por isso

Regras no papel **esquecem-se**: alguém novo, pressa, "só desta vez" — e em seis meses voltámos ao esparguete.

O Sheriff: o xerife da cidade

Bairros, crachás e uma lei — verificados automaticamente



Bairros

as nossas pastas

Crachás

as tags de cada pasta

A lei

as dependency rules

"Quem tem o crachá **ui** não entra em bairros **feature**."

O que é — e o que não é

É…

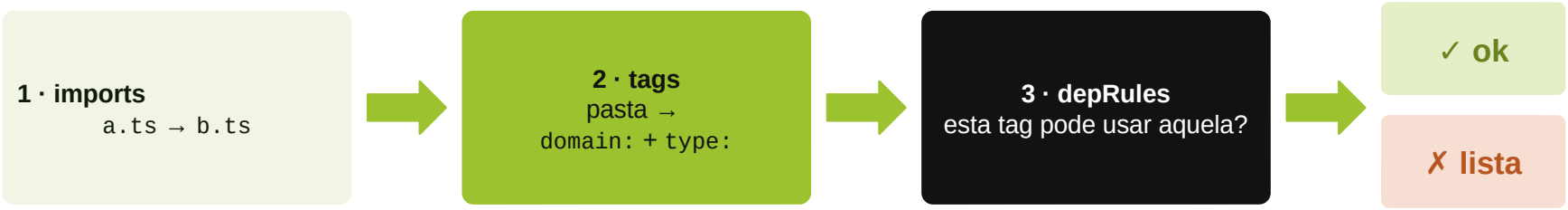
- Uma ferramenta **open source** que lê os imports
- Compara cada import com **as regras que escrevemos**
- Corre como um **teste**: no CI, o build falha

Não é…

- Um framework: **não muda a app** (só `devDependency`, fora do bundle)
- Uma reorganização: descreve as **pastas que já temos**
- Quem decide: **nós** escrevemos as regras

O TypeScript verifica os **tipos**, o ESLint o **estilo** — o Sheriff verifica as **fronteiras**.

Como funciona o Sheriff




```
src/app/domains/ticketing/ui/flight-card/flight-card.ts
  tags: domain:ticketing, type:ui
  importa → feature-booking/flight-search/flight-store.ts
  tags: domain:ticketing, type:feature
  type:ui → type:feature ?   ✗  Dependency Rule Violation
```

Na linha de comandos (`npx sheriff verify`, dá para CI) ou no ESLint (erros no editor).

Sheriff: as regras verificadas

sheriff.config.ts

```ts
export const config: SheriffConfig = {
  enableBarrelLess: true,
  modules: {                          // pastas → tags
    'src/app/domains/<domain>': {
      'feature-<name>': ['domain:<domain>', 'type:feature'],
      ui: ['domain:<domain>', 'type:ui'],  …
    },
  },
  depRules: {                         // tag → o que pode usar
    root: '*',
    'domain:*': [sameTag, 'domain:shared'],
    'type:feature': ['type:ui', 'type:data', 'type:util'],  …
  },
};
```

Corre…

- `npx sheriff verify` (linha de comandos, CI)
- `npx sheriff list` (que tags tem cada pasta)
- ou no **ESLint** (erros no editor)



Snippet A1 · da teoria ao código

Agora, vamos escrever o código

- `npm i -D @softarc/sheriff-core`
- Criar o `sheriff.config.ts` na raiz: as pastas (tags) e as regras
- Experimentar: `npx sheriff list` — 8 pastas, cada uma com o seu crachá

package.json · sheriff.config.ts (raiz do projeto)

A1 · Instalar e descrever a matriz

```
npm i -D @softarc/sheriff-core
```

sheriff.config.ts (raiz do projeto)

```ts
import { sameTag, SheriffConfig } from '@softarc/sheriff-core';
export const config: SheriffConfig = {
  entryFile: 'src/main.ts',
  enableBarrelLess: true,
  modules: {
    'src/app/domains/<domain>': {
      'feature-<name>': ['domain:<domain>', 'type:feature'],
      'ui-<name>': ['domain:<domain>', 'type:ui'],
      ui: ['domain:<domain>', 'type:ui'],
      data: ['domain:<domain>', 'type:data'],
      'util-<name>': ['domain:<domain>', 'type:util'],
    },
  },
  depRules: {
    root: '*',
    'domain:*': [sameTag, 'domain:shared'],
    'type:feature': ['type:ui', 'type:data', 'type:util'],
    'type:ui': ['type:data', 'type:util'],
    'type:data': ['type:util'],
    'type:util': [],
  },
};
```

Experimentar

- `npx sheriff list`
- 8 pastas, cada uma com `domain:` e `type:`



Snippet A2 · da teoria ao código

Agora, vamos escrever o código

- `npx sheriff verify` — o que encontra no Skyboard?
- Uma violação que já lá estava: discutir com a sala
- Decidir: ajustar a regra `'type:util'` → `No issues found`

terminal · sheriff.config.ts

A2 · O que o Sheriff encontra

```
$ npx sheriff verify

Issues found:
  Total Dependency Rule Violations: 1

|-- src/app/domains/shared/util-auth/auth-interceptor.ts
|   |-- Dependency Rule Violations
|   |   |-- from tag type:util to tags domain:shared, type:util
```

Já lá estava: util-auth → util-common

- O `authInterceptor` lê o `apiUrl` do `ConfigService`
- O que fazemos? Mover · passar de outra forma · **ajustar a regra**

```
// a equipa decide:
'type:util': ['type:util'],
```



Snippet A3 · da teoria ao código

Agora, vamos escrever o código

- No `FlightCard`, importar a `FlightStore` (só o import)
- `npx sheriff verify` → `type:ui` → `type:feature` ✗
- Desfazer o import

ticketing/ui/flight-card/flight-card.ts

A3 · Provocar uma violação

```
// ticketing/ui/flight-card/flight-card.ts — só para ver o Sheriff a reagir
import { FlightStore } from '../../feature-booking/flight-search/flight-store';
```

```
$ npx sheriff verify
|-- src/app/domains/ticketing/ui/flight-card/flight-card.ts
|   |   |-- from tag type:ui to tags domain:ticketing, type:feature
```

Porque é grave?

O `FlightCard` deixava de ser reutilizável: ficava preso a **um** caso de uso.

E também…

Importar algo de `data/internal/` → **Encapsulation Violation**.



Snippet A4 · da teoria ao código

Agora, vamos escrever o código

- Mover o `logging-interceptor.ts` para `shared/util-common/`
- Corrigir o import no `app.config.ts`
- Script `lint:arch` no `package.json` → `npm run lint:arch`

shared/util-common/logging-interceptor.ts · app.config.ts · package.json

A4 · Arrumar e automatizar

```
git mv src/app/domains/shared/ui-common/logging-interceptor.ts \
       src/app/domains/shared/util-common/
```

```
// app.config.ts
import { loggingInterceptor } from
  './domains/shared/util-common/logging-interceptor';
```

```
// package.json → scripts
"lint:arch": "sheriff verify"

$ npm run lint:arch   → No issues found.
```

O Sheriff verifica as **regras**; o **nome** da pasta continua a ser uma decisão nossa.

Um interceptor em `ui-common` não viola nenhuma regra (ui pode usar util) — mas engana quem procura.

Na vida real

Path mappings · ESLint · Detective

```
// tsconfig.json → compilerOptions
"paths": { "@skyboard/*": ["./src/app/domains/*"] }

// antes
import { FlightClient } from '../../data/flight-client';
// depois: domínio + módulo à vista
import { FlightClient } from '@skyboard/ticketing/data/flight-client';
```

ESLint

`@softarc/eslint-plugin-sheriff`: as violações aparecem **no editor**. Exige ESLint (`ng add angular-eslint`).

Detective

`npx detective`: o grafo de dependências no browser. Bom para conhecer um projeto novo.

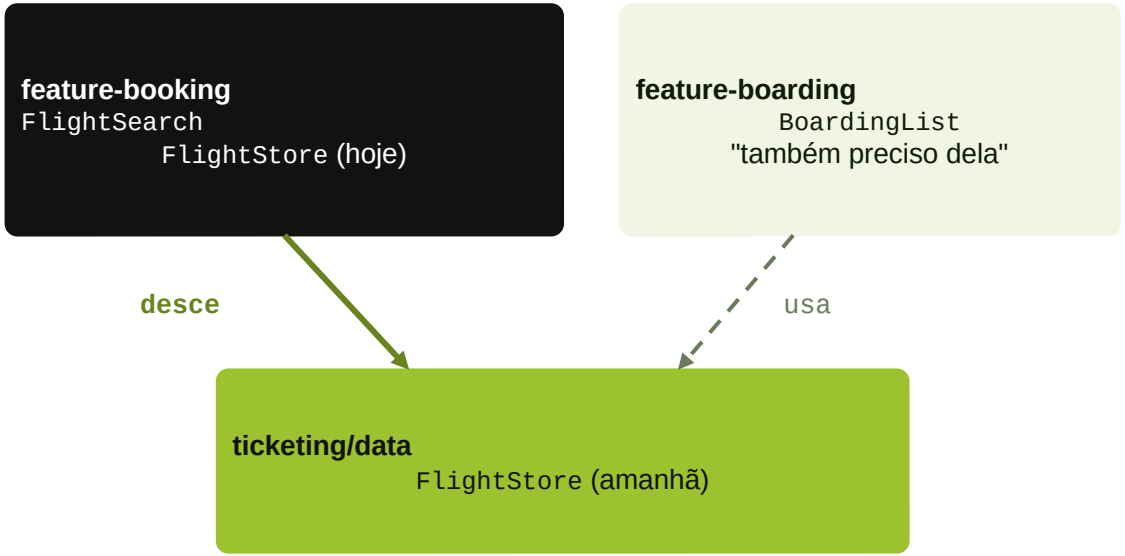


Bloco D

Local ou partilhado?

Feature slicing · stores · modulith

Feature slicing: tão local quanto possível



Regra prática

- Começa na feature
- Uma 2.ª feature precisa → desce para `ui` ou `data`
- Exige uma equipa atenta ao momento de refatorar

Tão local quanto possível, tão global quanto necessário.

Stores e arquitetura



Um só sentido: intenção → processar → estado → vista

Onde vive?

- **feature**: o caso de uso (a nossa `FlightStore`)
- **data**: partilhada no domínio
- **util**: genérica (a `AuthStore`)
- Começa local; desce quando outra feature precisar

Stores a falar com stores?

- Acoplamento e ciclos
- Só ler de uma camada **abaixo**
- Ou uma **facade** que orquestra
- Ou eventos → NgRx (Sessão 13)

Modulith ou micro frontends?

Modulith

- Uma codebase, módulos, **um** deploy
- Fronteiras protegidas por linting
- Fácil partilhar e refatorar
- A melhor escolha para 1–poucas equipas

Micro frontends

- Várias codebases, deploy **independente**
- Autonomia, até stacks diferentes
- Partilhar é mais difícil · mais infraestrutura
- Muitas equipas, sistemas grandes, anos

O Skyboard é um **modulith**.

Resumo da sessão

Domains × layers

- Cortar pelo negócio
- feature · ui · data · util
- Cada célula é uma pasta

Dependency rules

- Só para baixo
- O próprio domain + `shared`
- `internal/` = privado

Sheriff

- `sheriff.config.ts`: tags + `depRules`
- `npm run lint:arch` (CI)
- Exceções = decisões escritas

A pasta diz **o que a peça é** — e as dependency rules dizem **quem a pode usar**.



Na Sessão 13

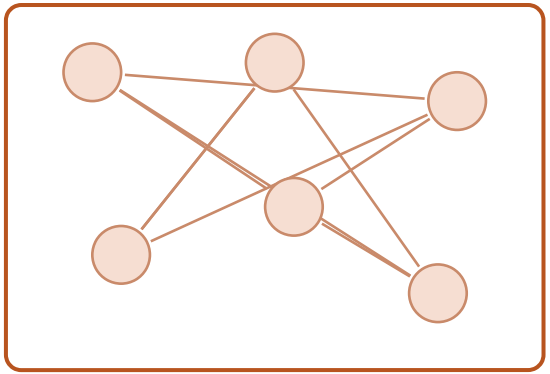
NgRx Signal Store

- A `FlightStore` com `withState` · `withComputed` · `withMethods`
- `rxMethod` · `toSignal` · `toObservable`
- Onde vive cada store — as dependency rules de hoje

Extra · Do monólito ao modulith

Fronteiras e deploys: três formas de organizar uma app

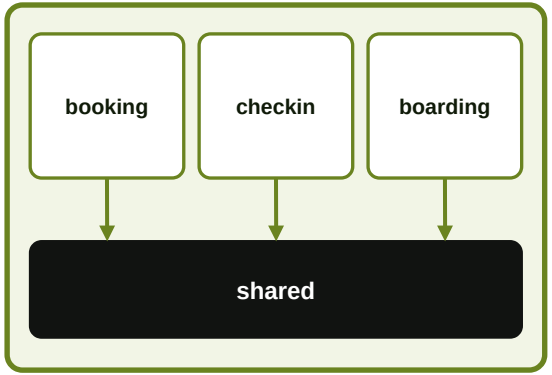
Big ball of mud



Fronteiras: **nenhumas**

Deploy: **um**

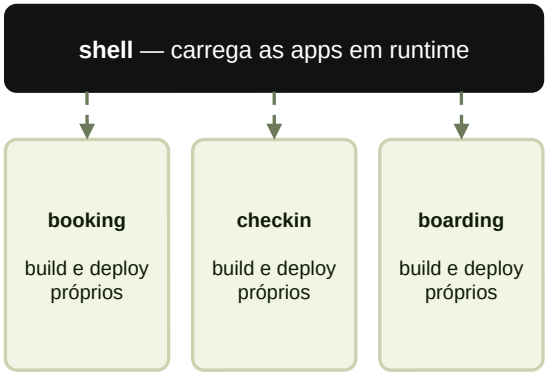
Modulith



Fronteiras: **claras e verificadas**

Deploy: **um**

Micro frontends



Fronteiras: **físicas**

Deploys: **vários**

O problema nunca foi ter **um só deploy** — foi **não ter fronteiras**.

O Skyboard é um modulith. Um módulo bem isolado extrai-se mais tarde para micro frontend, sem reescrever.

Extra · Monorepo e micro frontends

Onde guardas o código ≠ como o entregas

Monorepo: onde está o código

- Um repo, várias apps e libraries
- Partilhar e refatorar fácil · uma versão de cada dependência
- **Nx**, Turborepo: build só do que mudou, cache
- Fronteiras: `enforce-module-boundaries` (como o Sheriff)

Micro frontends: como se entrega

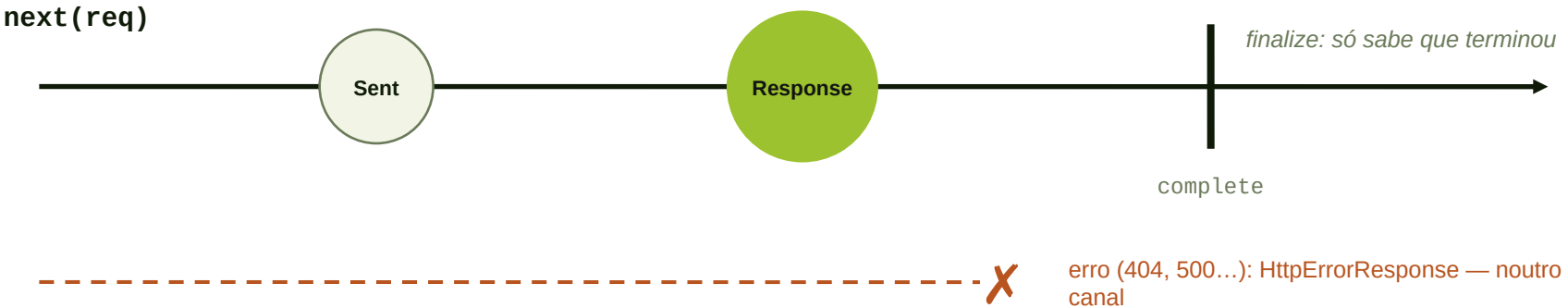
- Várias apps, deploys independentes, uma shell
- Autonomia total por equipa
- Custo: infraestrutura, versões, partilha difícil
- Angular: **Module Federation** · **Native Federation**

	Um deploy	Vários deploys
Um repo	Modulith num monorepo — ou só uma app (o Skyboard)	Micro frontends num monorepo (comum com Nx)
Vários repos	Raro	Micro frontends "clássicos" (polyrepo)

Monorepo é sobre **onde guardas** o código; micro frontends, sobre **como o entregas**.

Extra · Pergunta da Sessão 10: ler a response

O next(req) emite vários eventos — não só a response



Ler

`tap` + filtrar `HttpEventType.Response`

Alterar

`map` + `event.clone({ body })`

Erros

`tap({ error })` ou `catchError`



Snippet X1 · da teoria ao código

Agora, vamos escrever o código

- No `loggingInterceptor`: trocar o `return` — `tap` para ler a response
- Filtrar o evento `Response` (o `next(req)` emite vários)
- Experimentar: abrir um voo e fazer Save — ver o body na consola

shared/ui-common/logging-interceptor.ts

Extra · Pergunta da Sessão 10: ler a response

No loggingInterceptor

```
return next(req).pipe(
  tap((event) => {
    // de todos os eventos, só nos interessa a response
    if (event.type === HttpEventType.Response) {
      console.log(`← ${event.status}`, event.body);
    }
  }),
  finalize(() => { /* … os ms, como antes */ }),
);
```

Notar

- `next(req)` emite vários `HttpEvent`: filtrar o `Response`
- Ler: `tap` · alterar: `map` + `clone`
- Errors chegam à parte: `HttpErrorResponse`

O `finalize` não recebe valor: só sabe que o request **terminou**. O `body` é `unknown`.